Umelé neurónové siete – Klasifikácia na medicínskych dátach

Obsah

Úloha 6: Klasifikácia na medicínskych dátach **2**
Cieľ úlohy . . . 2
Definícia úlohy . . . 2
Postup riešenia . . . 3

Odovzdanie a hodnotenie **4**
Dokumentácia výsledkov (PDF) . . . 4
Obhajoba riešenia . . . 4

Odporúčané nástroje a technická realizácia **5**
Matlab . . . 5
Python . . . 5

Teoretické poznámky **6**
Náhodnosť pri trénovaní a férové porovnávanie modelov . . . 6
Krížová validácia a opakované spustenia trénovania . . . 6
Metriky pri medicínskej klasifikácii . . . 7
Kapacita siete, generalizácia a pretrénovanie . . . 8

Úloha 6: Klasifikácia na medicínskych dátach pomocou MLP siete (4b)

Cieľ úlohy

Vytvorte a natrénujte neurónovú sieť (ANN/UNS), ktorá bude schopná klasifikovať stav bábätka na základe CTG vyšetrenia(*Kardiotokografia*).
Úlohou siete je zatriediť kardiozáznam do **3 tried**:

1. **normálny**
2. **podozrivý**
3. **patologický**

Štruktúru siete a parametre trénovania nastavte tak, aby ste dosiahli čo najlepšiu percentuálnu úspešnosť klasifikácie na testovacích dátach. Pre **najlepšiu z porovnávaných štruktúr** musí byť **priemerná** percentuálna úspešnosť klasifikácie na testovacích dátach **väčšia ako 92%**, pričom priemer sa určí z **5 spustení** pri náhodnom rozdeľovaní dát alebo z **krížovej validácie**.

Definícia úlohy

CTG je vyšetrenie, ktoré zaznamenáva ozvy srdca bábätka a sťahy maternice. Z kardiozáznamu boli vypočítané vybrané parametre (príznaky), na základe ktorých boli záznamy zatriedené do tried. Pre klasifikáciu boli vytvorené dáta z **2126 meraní**, z ktorých bolo vypočítaných **25 parametrov**.

V Matlabe sa všetky dáta nachádzajú v súbore `CTGdata.mat`, kde vstupné parametre sú v premennej `NDATA` a skupiny ochorenia v premennej `typ_ochorenia`. Pri realizácii v Pythone použite súbor `CTGdata.csv`, v ktorom prvých **25 stĺpcov** (označených `0` až `24`) predstavuje vstupné parametre a posledný stĺpec `typ_ochorenia` obsahuje cieľovú triedu.

Postup riešenia

1. Použite dátový súbor `CTGdata.mat`. Pri riešení v Pythone použite dátový ekvivalent `CTGdata.csv`.
2. Vstupné dáta predstavujú **25 parametrov** vypočítaných z meraných CTG signálov. Výstupom siete je zaradenie vzorky do **3 tried** (`1 - normálny`, `2 - podozrivý`, `3 - patologický`).
3. Na trénovanie použite maximálne **60%** z celkového počtu dát. Zvyšné dáta použite na testovanie a prípadne validáciu.
4. Navrhnite MLP sieť a nastavte parametre trénovania tak, aby ste dosiahli čo najlepšiu percentuálnu úspešnosť klasifikácie na testovacích dátach.
5. Najprv nájdite takú štruktúru siete, ktorá spĺňa požiadavky klasifikácie. Pri porovnaní vyskúšajte ešte **aspoň dve ďalšie štruktúry**, teda spolu minimálne **3 rôzne štruktúry** siete (napr. rozdielny počet neurónov v skrytej vrstve alebo odlišnú hĺbku siete).
6. Pri porovnávaní jednotlivých štruktúr zachovajte **rovnaké hyperparametre** trénovania, najmä počet epoch, learning rate, batch size a ostatné hlavné nastavenia. Výnimkou môže byť iba **early stopping**, ak ho v riešení použijete.
7. Pre každú porovnávanú štruktúru vykonajte **krížovú validáciu**, alebo ju pri náhodnom rozdeľovaní dát **spustite minimálne 5-krát**. Ak nepoužijete krížovú validáciu, pri minimálne 3 štruktúrach to znamená celkovo spolu aspoň **15 behov**.
8. Pre **najlepší výsledok z každej porovnávanej štruktúry** zdokumentujte proces trénovania, výslednú kontingenčnú maticu a dosiahnuté úspešnosti klasifikácie na **trénovacích**, **testovacích** a **celkových** dátach.
9. Pre každú porovnávanú štruktúru vyčíslite **minimálnu**, **maximálnu** a **priemernú** úspešnosť klasifikácie na trénovacích a testovacích dátach pri krížovej validácii alebo pri 5 spusteniach s náhodným rozdelením dát.
10. Pre najlepšie natrénovanú sieť otestujte z každého typu ochorenia aspoň jednu vzorku z datasetu. Vyčíslite výstup siete a zatrieďte vzorky do skupín.
11. Pre najlepšie natrénovanú sieť vypočítajte aj **senzitivitu** a **špecificitu**.

Poznámka: Pri porovnávaní štruktúr siete je cieľom porovnávať **porovnateľné nastavenia**. Ak sa mení štruktúra siete, ostatné hlavné hyperparametre by mali zostať rovnaké, aby bolo možné výsledky medzi modelmi férovo porovnať. Ak používate validačné dáta, graf **loss vs. epoch** zobrazujte ako spoločný graf trénovacej a validačnej chyby. Na testovacích dátach sa potom uvádzajú už len výsledné metriky po natrénovaní siete.

Zároveň sa odporúča, aby boli výsledky v dokumentácii spracované nielen vo forme grafov, ale aj v **prehľadných tabuľkách**. Najmä použité hyperparametre, porovnávané štruktúry siete a výsledky jednotlivých spustení pre danú štruktúru by mali byť uvedené tabuľkovo, aby bolo porovnanie čitateľné, jednoznačné a malo primeranú technickú a inžiniersku formu. Nie je vhodné nahrádzať takýto prehľad iba screenshotmi výstupov alebo priamym kopírovaním konzolového výpisu bez ďalšieho spracovania.

Odovzdanie a hodnotenie

Hodnotenie zadania prebieha formou osobnej konzultácie. Zadanie sa odovzdáva vždy na **začiatku cvičenia** po odprezentovaní nasledujúceho zadania.

Počas semestra môže študent nahradiť najviac dve cvičenia. V takom prípade je možné zadanie odovzdať aj na cvičení, na ktorom študent nahrádza. V ostatných prípadoch sa zadanie odovzdáva vždy svojmu cvičiacemu.

Všetky požadované výsledky musia byť pripravené **už na začiatku cvičenia** a uvedené v dokumentácii. Opätovné spúšťanie trénovania počas obhajoby sa vykonáva len v prípade, že o to cvičiaci výslovne požiada.

Dokumentácia výsledkov (PDF)

Pripravte stručnú správu (PDF), ktorá obsahuje grafy z viacerých spustení a krátku analýzu vplyvu zvolených parametrov. Pri neurónových sieťach typicky uvádzajte priebeh učenia ako **loss vs. epoch** a podľa typu úlohy aj vhodné metriky. Dokument slúži ako podklad pre diskusiu, aby nebolo nutné čakať na opakované výpočty počas obhajoby.
Pre toto zadanie uveďte v správe:

1. stručný opis úlohy, použitých dát a spôsob ich rozdelenia na trénovacie, testovacie a prípadne validačné dáta
2. popis aspoň **3 porovnávaných štruktúr** MLP siete a použitých hyperparametrov, pričom pri porovnaní jednotlivých štruktúr použite **rovnaké hlavné hyperparametre** trénovania
3. ak nepoužijete krížovú validáciu, potom pre **každú** z porovnávaných štruktúr vykonajte **minimálne 5 behov**, teda pri minimálne 3 štruktúrach ide **celkovo aspoň o 15 spustení** pre celé zadanie
4. pre **každú štruktúru** uveďte minimálnu, maximálnu a priemernú úspešnosť klasifikácie a pre **najlepší výsledok z každej štruktúry** uveďte aj priebeh učenia a kontingenčnú maticu
5. postup testovania vybraných vzoriek z datasetu a pre **najlepšie natrénovanú sieť** aj výslednú úspešnosť klasifikácie, senzitivitu a špecificitu.

Súčasťou dokumentácie **musia byť aj prehľadné tabuľky** so zvolenými hyperparametrami, porovnávanými štruktúrami siete a výsledkami jednotlivých behov.Ukážky vhodného obsahu tabuliek sú uvedené nižšie v Tab. 1, Tab. 2 a Tab. 3.

Obhajoba riešenia

Súčasťou hodnotenia je demonštrácia funkčného kódu a vysvetlenie implementácie. Body sa udeľujú najmä za porozumenie princípu algoritmu a schopnosť zdôvodniť dosiahnuté výsledky, nie iba za spustiteľný program.

V diskusii buďte pripravení interpretovať grafy učenia (**loss vs. epoch** a prípadné metriky) a zdôvodniť vplyv nastavení (napr. architektúra, regularizácia, learning rate, batch size) na konvergenciu a opakovateľnosť výsledkov.

Pri obhajobe vysvetlite popis vstupných a výstupných dát, zvolenú štruktúru MLP siete, nastavené parametre trénovania a rozdiely medzi porovnávanými štruktúrami siete. Na základe grafov učenia a kontingenčných matíc zdôvodnite, prečo bola vybraná najlepšia sieť a ako sa prejavilo náhodné rozdelenie dát na výslednej úspešnosti klasifikácie.

Zároveň vysvetlite, prečo sa pri porovnávaní modelov používa **krížová validácia** alebo **opakované spúšťanie**, a prečo nestačí hodnotiť model iba z jedného náhodného rozdelenia dát.

Odporúčané nástroje a technická realizácia

Zadanie je možné vypracovať v prostredí **Matlab** alebo v jazyku **Python**. Ako alternatívu bez inštalácie je možné použiť **MATLAB Online** (`https://matlab.mathworks.com/`). Pre Python odporúčame **Google Colab** (`https://colab.research.google.com/`).

Matlab

Pri Matlabe použite vhodné nástroje pre neurónové siete podľa pokynov na cvičení. Do správy zahrňte priebeh učenia (loss vs. epoch) a výsledné metriky podľa zadania.

Pri Matlabe použite súbor `CTGdata.mat`. Vstupné dáta sa nachádzajú v premennej `NDATA` a cieľové triedy v premennej `typ_ochorenia`. Na klasifikáciu použite vhodnú MLP sieť podľa pokynov na cvičení.

Python

Pri Pythone odporúčame pracovať v prostredí Google Colab. Pre úlohy v tomto bloku použite oficiálne repozitáre:

- **Dodatočné kódy a zadania:** `https://github.com/STU-FEI-OUI/UMINT-UNS`
- **Dáta pre Python:** `https://github.com/STU-FEI-OUI/UMINT-UNS_data`

Pre vizualizáciu uveďte aspoň priebeh učenia ako **loss vs. epoch** a podľa zadania doplňte validáciu a metriky.

Pre Python použite súbor `CTGdata.csv` z repozitára **UMINT-UNS_data**. Ukážka načítania dát priamo z GitHub raw URL:

```
import pandas as pd
url = "https://raw.githubusercontent.com/STU-FEI-OUI/UMINT-UNS_data/main/Python_(CSV)/CTGdata.csv"
df = pd.read_csv(url)
##print(df.head())
```

Pri implementácii je vhodné doplniť aj jednoduchý **early stopping**, napríklad podľa validačnej chyby. Ak validačná chyba počas zvoleného počtu epoch už neklesá alebo sa začne zhoršovať, tréning sa môže predčasne ukončiť a ponechá sa najlepšia verzia siete pred pretrénovaním.

Teoretické poznámky

Náhodnosť pri trénovaní a férové porovnávanie modelov

Pri trénovaní neurónových sietí môže výsledok ovplyvniť viacero náhodných faktorov, napríklad počiatočná inicializácia váh, náhodné rozdelenie dát alebo poradie vzoriek počas učenia. Aj pri rovnakej architektúre siete a pri rovnakých hyperparametroch preto nemusia dve spustenia viesť k úplne rovnakému výsledku. Práve z tohto dôvodu sa pri porovnávaní modelov nesleduje iba jeden beh, ale stabilita výsledkov vo viacerých behoch.

Aby bolo porovnanie rôznych štruktúr siete férové, je potrebné zachovať čo najviac **rovnaké nastavenia trénovania**. Ak by sa spolu so štruktúrou menili aj learning rate, batch size, počet epoch alebo ďalšie hyperparametre, nebolo by jasné, či rozdiel vo výsledku spôsobila samotná štruktúra siete alebo iné nastavenie učenia. Pri porovnávaní architektúr preto má zmysel meniť najmä samotnú sieť a ostatné hlavné parametre ponechať rovnaké, prípadne odchýlky jasne zdôvodniť.

Na poznámky z predchádzajúceho zadania o delení datasetu na trénovacie, validačné a testovacie dáta možno nadviazať aj tu. Najmä pri hodnotení viacerých štruktúr je dôležité, aby bolo zrejmé, ktoré dáta slúžia na učenie, ktoré na ladenie modelu a ktoré zostávajú vyhradené na finálne overenie výsledku.

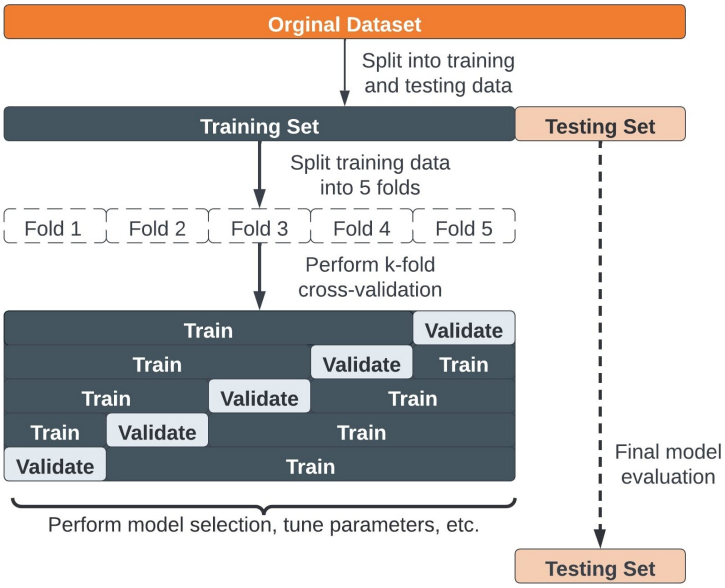
Krížová validácia a opakované spustenia trénovania

Pri medicínskych dátach nestačí hodnotiť sieť iba podľa jedného náhodného rozdelenia dát. Jedno rozdelenie môže byť náhodne výhodnejšie alebo nevýhodnejšie, a preto by neposkytovalo dostatočne spoľahlivý obraz o kvalite modelu. Z tohto dôvodu sa používa buď **krížová validácia**, alebo **opakované spustenia trénovania**.

Pri **krížovej validácii** sa najprv pôvodný dataset rozdelí na **trénovaciu** a **testovaciu** časť. **Testovacia množina** zostáva počas výberu modelu bokom a použije sa až na **finálne vyhodnotenie**. Len **trénovacia množina** sa ďalej rozdelí na viac častí, tzv. **foldov**. Fold je teda jedna podmnožina trénovacích dát, ktorá sa v niektorom kroku použije na validáciu, zatiaľ čo ostatné foldy slúžia na trénovanie. Ak sa použije napríklad 5-fold cross-validation, potom prebehne **5 iterácií**, pričom v každej iterácii sa jeden fold použije na validáciu a ostatné štyri na trénovanie. V ďalšej iterácii sa validačný fold vymení. Takto sa postupne vystriedajú všetky foldy a výsledok sa vyhodnotí z celej série behov. Tento princíp je schematicky znázornený na Obr. 1.

Pri **opakovaných spusteniach trénovania** sa naopak ponechá **rovnaká finálna testovacia množina** a model sa učí viackrát odznova. V jednotlivých behoch sa mení najmä náhodné rozdelenie dát určenej tréningovej časti na trénovaciu a validačnú množinu, prípadne aj náhodné poradie vzoriek počas učenia a počiatočná inicializácia siete. Testovacie dáta sa do trénovania nezapájajú a zostávajú počas všetkých behov rovnaké. Tento prístup je jednoduchší na implementáciu a zároveň ukazuje, nakoľko je výsledok citlivý na konkrétny náhodný výber a priebeh učenia.

Oba prístupy majú spoločný cieľ: nespoliehať sa na jeden náhodný experiment, ale získať spoľahlivejší obraz o správaní modelu. Pri interpretácii výsledkov preto má význam sledovať **minimálnu**, **maximálnu** a **priemernú** úspešnosť klasifikácie. Maximum ukazuje najlepší dosiahnutý výsledok, minimum zasa najhorší prípad a priemer poskytuje obraz o tom, ako sa model správa typicky.



Obr. 1: Schéma 5-fold krížovej validácie.

Pôvodný dataset sa najprv rozdelí na trénovaciu a finálnu testovaciu množinu. Len trénovacia množina sa ďalej rozdelí na 5 foldov, ktoré sa v jednotlivých iteráciách striedajú v úlohe validačnej časti. Finálna testovacia množina zostáva počas výberu modelu oddelená a použije sa až na záverečné vyhodnotenie.

Metriky pri medicínskej klasifikácii

V predchádzajúcom zadaní už boli spomenuté základné metriky učenia a klasifikácie. Pri medicínskych dátach je však dôležité venovať väčšiu pozornosť tomu, **aké typy chýb** model robí, nie iba tomu, koľko vzoriek klasifikuje správne celkovo. Samotná **accuracy** je síce užitočný orientačný ukazovateľ, ale nemusí vždy dostatočne vystihnúť kvalitu modelu, najmä ak majú jednotlivé triedy rozdielny význam.

Preto sa pri klasifikačných úlohách používa aj **kontingenčná matica**, ktorá ukazuje, medzi ktorými triedami si sieť robí chyby. Pri medicínskej klasifikácii má navyše zmysel sledovať aj **senzitivitu** a **špecificitu**, pretože tieto ukazovatele lepšie odrážajú, ako dobre model zachytáva pozitívne a negatívne prípady po zvolení príslušného rozdelenia tried. V medicínskych úlohách býva totiž často dôležitejšie správne rozpoznať rizikové alebo podozrivé prípady, než len maximalizovať jedno celkové číslo úspešnosti.

		Predicted class: Positive	Predicted class: Negative	
Actual class	Positive	True Positive (TP)	False Negative (FN) Type II Error	**Recall** $\frac{TP}{TP+FN}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	**Specificity** $\frac{TN}{TN+FP}$
		Precision $\frac{TP}{TP+FP}$	**Negative predictive value** $\frac{TN}{TN+FN}$	**Accuracy** $\frac{TP+TN}{TP+TN+FN+FP}$

Obr. 2: Vzťah medzi kontingenčnou maticou a vybranými metrikami klasifikácie, najmä accuracy, precision, recall (senzitivita) a specificitou.

Na Obr. 2 je znázornené, ako sa z hodnôt **TP**, **TN**, **FP** a **FN** odvodzujú základné metriky klasifikácie. V tomto zadaní sú z praktického hľadiska dôležité najmä **senzitivita** (*recall*) a **špecificita**, pretože lepšie vystihujú správanie modelu pri rozlišovaní medzi negatívnymi a pozitívnymi prípadmi než samotná celková úspešnosť.

Pri hodnotení modelu preto treba rozlišovať medzi tým, čo je vhodné na **učenie modelu**, a tým, čo je vhodné na **interpretáciu výsledkov**. Loss slúži najmä na optimalizáciu siete počas trénovania, zatiaľ čo accuracy, confusion matrix, senzitivita a špecificita poskytujú čitateľnejší pohľad na to, ako sa model správa z hľadiska praktického použitia.

Kapacita siete, generalizácia a pretrénovanie

Pri porovnávaní viacerých štruktúr siete nejde len o to, ktorá dosiahne najvyššiu úspešnosť v jednom konkrétnom behu. Dôležitá je aj **generalizácia**, teda schopnosť siete fungovať rozumne aj na dátach, ktoré počas učenia nevidela. Na túto tému už nadväzujú poznámky z predchádzajúceho zadania o delení datasetu, validačných dátach a o pretrénovaní.

Vo všeobecnosti platí, že sieť s väčšou kapacitou môže lepšie zachytiť zložité vzťahy v dátach, ale zároveň sa môže ľahšie prispôsobiť aj náhodným odchýlkam alebo šumu v tréningovej množine. Menšia sieť môže byť naopak stabilnejšia, ale nemusí mať dostatočnú schopnosť zachytiť všetky potrebné vzťahy. Preto sa v tomto zadaní neporovnáva len najlepší výsledok, ale aj správanie viacerých štruktúr pri rovnakých podmienkach.

V tejto súvislosti môže byť užitočný aj **early stopping**, teda predčasné ukončenie trénovania pri zhoršovaní validačnej chyby. Jeho úlohou nie je „zázračne zlepšiť“ model, ale obmedziť situácie, v ktorých sa sieť začne príliš prispôsobovať tréningovým dátam. Aj pri použití early stoppingu však stále platí, že výsledok treba posudzovať na nezávislých testovacích dátach a ideálne vo viacerých behoch alebo pri krížovej validácii.

Model	Skryté vrstvy	Neuróny	Epochy	LR	Batch
M1	1	10	300	0.001	32
M2	1	20	300	0.001	32
M3	2	20,10	300	0.001	32

Tabuľka 1: Porovnávané štruktúry siete a hlavné hyperparametre.

Beh	Train loss	Test loss	Train acc [%]	Test acc [%]
1	0.082	0.114	95.8	93.1
2	0.076	0.121	96.2	92.7
3	0.085	0.108	95.5	93.8
4	0.079	0.117	96.0	92.9
5	0.081	0.112	95.9	93.4

Tabuľka 2: Výsledky 5 spustení pre štruktúru M2(M1-M3).

Model	Min test acc [%]	Max test acc [%]	Priemer test acc [%]	Priemer test loss
M1	91.8	93.0	92.4	0.126
M2	92.7	93.8	93.2	0.114
M3	91.9	93.5	92.8	0.119

Tabuľka 3: Súhrnné porovnanie porovnávaných štruktúr.